

Continuous fragment growth: literature and mechanism assessment

The frozen continuous-fragment-growth algorithm has a defensible, specific contribution in its combined search policy and state representation. No equivalent of its complete transition rule was identified in the primary methods and implementation sections examined. The strongest claim concerns collective fragment commitment across retained placements, saturation-level whole-mapping branching, whole-island absorption, and continuation joins that preserve future-relevant context. It does not concern the first use of growing fragments, multiple seeds, branching, canonicalization, bond weights, or molecular symmetry.

The distinction is substantive: the algorithm can make different decisions and retain different alternatives from the closest inspected procedures. Establishing that distinction supports describing an original specialized atom-mapping algorithm. It does not establish that no equivalent procedure exists in inaccessible publications, proprietary implementations, or adjacent fields.

Evidence scope

The comparison concerns the stable benchmark revision 98b01b175eed31f70d13e7cbf178b80bf07c9e0, including its Python and native growth loops. Later acceleration changes are outside this assessment. Source inspection establishes the implemented transitions; it does not establish complete enumeration, global optimality, or a new performance result.¹

Five reviews or comparative overviews provide coverage across historical reaction mapping, common-subgraph algorithms, optimization formulations, and recent mapping systems: Chen, Chen and Taylor (2013); Akutsu and Nagamochi (2013); Duesbury, Holliday and Willett (2017); Preciat-Gonzalez and colleagues (2017); and Lin and colleagues (2022). Relevant algorithm sections and citation trails were examined, rather than treating their bibliographies as evidence that every cited method had been inspected.^{2,3,4,5,6}

The accompanying coverage ledger distinguishes original method sections, implementation inspection, abstract-only coverage, and unresolved historical leads. The query log records the additional discovery searches. This is a broad review-led assessment with explicit gaps, not a reproducible census of every record in all bibliographic databases. Recent primary sources and pinned implementations extend coverage into 2026.

The mechanism that requires comparison

A local growth state contains a common source fragment F , retained compressed placements C , a frontier heap Q , an existing mapping L , its source-island partition I , deferred edges D , and processed-edge bookkeeping. The heap selects the strongest currently exposed source WBO edge, with atom-index tie breaking. Newly exposed edges enter as growth proceeds; this is not simply one global ordering of all bonds.¹

For an eligible extension to atom x , the algorithm obtains supported children across every retained placement and contextually deduplicates them. If that collection exceeds the live cap, the subtree is capped. Otherwise, a nonempty collection commits the shared fragment extension and replaces the old placements with their supported children. Placements that cannot follow that extension disappear. If no placement supports the extension, the fragment and candidates remain, and the edge is recorded as deferred.

The decisive feature is collective commitment. If placements A and B both fit F , but only A supports the selected extension, the algorithm commits $F + x$ and drops B. It does not simultaneously preserve B on the smaller fragment as a branch that skips the successful extension. This creates a meaningful search tradeoff: a discarded placement might have supported a useful different fragment shape.

Growth continues after the candidate collection becomes unique and ends when the eligible frontier is exhausted, subject to caps. Local placement children can branch during extension; the whole-mapping branch split follows saturation. Reaching an already locked source island absorbs that entire island with its established mapping. Deferral records a failed frontier attempt; it does not delete that bond from the graph. Subsequent compatibility checks still consider active source edges connecting a new atom to the grown fragment.¹

The local representation also matters. A placement stores a witness, source blocks and target pools, fixed assignments, and automorphism-related metadata. Extension can search for an injective supporting rearrangement within affected blocks; it is not confined to testing the stored witness. The support search is bounded and is not a proof that every possible support survives every input and budget.⁷

Closest inspected growth procedures

The following comparisons concern actual control flow. The cited versions and named routines are the evidence boundary; they do not describe every historical release of each package.

Comparator	Inspected decision	Difference relevant to the contribution
OpenChemLib 2026.2.0	mapFromRootAtoms scores neighboring atom pairs, commits one selected pair immediately, and queues it. An outer search tries root-pair sequences and retains a best completed mapping.	A concrete placement grows locally. Reproducing the baseline requires retaining a collection of placements of one shared fragment and changing when whole-mapping alternatives split.
SynKit approximate MCS, pinned revision	_grow_from_seed takes a high-degree frontier vertex, skips it if unsupported, otherwise assigns the highest-scoring compatible target. Each seed has one mapping dictionary.	It has genuine frontier growth and failure skipping, but no collective support decision over multiple live placements of the same fragment.

Comparator	Inspected decision	Difference relevant to the contribution
RDKit documented Python FMCS	Enumerates alternative query subgraphs, tests their occurrence, and uses bounds toward maximum cardinality.	The baseline commits a supported extension without retaining a local alternative that omits it. It is not an exhaustive search over fragment shapes.
Indigo, pinned revision	<code>_handleWithProduct</code> tries substructure matching, falls back to MCS, commits returned atom pairs, removes mapped product atoms, and repeats. An outer loop tries reactant permutations.	The next operation is a new residual matching step. The inspected loop does not maintain the baseline's compressed placement collection during a common-fragment growth phase.
RDT, pinned revision	Its MAX loop selects winning molecule pairs, obtains a stored MCS atom map, commits it, removes the matched atoms, and updates matching matrices.	The inspected unit of commitment is a selected match on residual structures, rather than the baseline's collective frontier transition and saturation scheduler.

Sources for these rows are the inspected OpenChemLib growth routine, SynKit approximate matcher, FMCS documentation, Indigo automapper, and RDT mapping engine and graph-matching implementation.^{8,9,10,11,12}

SynKit's approximate routine is especially useful for a precise comparison because its surface description sounds close: seed selection, neighboring growth, compatibility tests, and optional pruning. Its optional `prune_automorphisms` path sorts completed partial maps by size and suppresses repeated target-vertex sets. That is a different object and time of comparison from the baseline's contextual live-placement deduplication. This observation concerns that routine, separately from SynKit's exact atom mapper.⁹

The RDT source examined is a contemporary repository revision, not a reconstruction of the 2016 publication. Likewise, the Indigo comparison is pinned to the inspected snapshot. These are valid implementation comparisons, but publication-year priority claims require the corresponding historical releases. The ledger preserves this distinction.

Historical fragment-growth leads

Duesbury and colleagues identify Armitage-Lynch (1967) as an early connected-subgraph enumeration method: surviving small common substructures are extended and filtered, with canonicalization used to recognize duplicates. Their account also points to Varkony and colleagues (1979), Takahashi and colleagues (1987), and later FMCS. This is an important historical lead because it places growth plus duplicate suppression well before contemporary atom mappers. The original Armitage-Lynch publication was identified, but its full algorithm text was unavailable in this assessment; the review account must not be presented as an independent inspection of that original.^{4,13}

Chen and colleagues additionally identify McGregor-Willett, Funatsu and colleagues, Akutsu, Crabtree-Mehta, and subsequent graph-naming improvements. These are priority checks for seeded MCS, repeated fragment extraction, cut enumeration, and canonical matching. The inaccessible originals remain open comparisons. Their existence prevents a broad first-ever fragment-growth claim; the review alone does not establish equivalence to the complete baseline transition.²

The direct FMCS comparison already exposes the main distinction from fragment-shape enumeration. Nevertheless, excluding all historical variants would require original methods for the closest unresolved growth papers. A bibliography entry, an abstract, and a review summary provide different levels of evidence and are recorded accordingly.

Branching, candidate retention, and stopping rules

McSplit maintains a concrete partial mapping and paired classes of possible future vertex assignments. Its recursion explores images of a selected source vertex and an alternative that leaves the vertex unmatched, with maximum-cardinality bounds. These classes are not a collection of complete placements of the already grown common fragment. The baseline's collective commitment and saturation-level whole-mapping split require changing both the represented state and the available transitions.¹⁴

Jaworski and colleagues describe bounded isomorphic-mapping search using progressively smaller atom environments and matching of immediate neighborhoods; remaining unmatched regions lead to bond-cut searches. That is a close atom-mapping comparison because it combines local structure preservation with a larger search. Its main article establishes these operations, but inaccessible supplementary detail limits finer claims about every pruning and duplicate-elimination step.¹⁵

gSpan extends graph patterns through their occurrences and uses minimum DFS codes to avoid redundant pattern generation. This provides a concrete adjacent precedent for retaining occurrence information while growing a shared pattern. Its search branches over child patterns in a frequent-subgraph task. The baseline instead commits a supported source extension under its AAM scheduler. A claim based only on shared-pattern growth with reused occurrences would therefore be too broad.¹⁶

Petersen and colleagues' 2025 multiple-MCS method retains maximal intermediate common subgraphs and uses modular products with connected-clique enumeration. Its candidate retention serves preservation of maximum common structures across additional input graphs. Those candidates, and the additional-graph transition, differ from placements of one growing source fragment within a reaction. The original methods sections support that distinction; this assessment does not independently verify the paper's proofs or implementation.¹⁷

Two contemporary AAM implementations provide further direct controls. SLAP solves label-conditioned linear assignment problems, refines paired label states, and suppresses repeated label states. SynKit's exact mapper recursively assigns uncertainty-kernel atoms with cost bounds and symmetry-related pruning. Neither inspected transition is the common-fragment accept/defer loop. Both already branch and eliminate repeated work, so those generic properties cannot differentiate the baseline.^{18,19}

Symmetry, compression, and continuation equality

Three different equivalence questions should remain separate. Local placement equivalence asks whether candidates can be represented together while preserving growth support. Continuation equality asks whether two histories have the same future under the scheduler. Final-family equality asks whether completed records describe the same correlated assignments. Success at one level does not automatically answer the others.

The baseline's local canonicalization incorporates source identities, pools, locked images, and boundary information in the active graph context. Its whole-mapping join compares exact assignments, a normalized source-island partition, and deferred edges at the same pass/seed position. A completed family instead uses a representative mapping with automorphism generators and joint assignment correlations.⁷

Including the island partition has an operational reason. Two histories can agree on every current atom assignment but divide source atoms into different committed islands. Reaching an atom in the next growth phase then absorbs different collections of locked atoms. Such histories cannot safely be declared interchangeable solely because the current atom-to-atom dictionary is equal. This is a distinguishing state requirement created by the whole-island transition.

The general idea of joining histories with identical future subproblems is established in context-based search, including AND/OR search graphs. Graph canonicalization and automorphism computation also have extensive prior foundations. The potentially original part is the specific future-relevant state and how it interacts with this scheduler, not the abstract idea of caching or quotienting a search.^{20,21}

Laffitte, Phan and Stadler's 2026 work is particularly relevant: it studies extensions of partial atom maps using structural constraints and automorphism relationships. Its extension setting and anchored isomorphism operations provide substantive precedent for symmetry-aware partial-map completion. The baseline comparison must therefore identify its collective frontier policy, island absorption, and continuation key explicitly instead of describing partial-map extension with symmetry as new.²²

Mann and colleagues provide a separate constraint-based precedent: cyclic reaction-center patterns constrain candidates, followed by graph-matching extension. Ali and colleagues enumerate common-edge-subgraph-derived mappings and reduce symmetry-related alternatives. These are important comparators for retained alternatives and symmetry, although their inspected generation procedures differ from the baseline's frontier transition.^{23,24}

Symmetry-aware learned mapping is also established. AMNet (2024) forms node embeddings and pairwise correspondence probabilities and incorporates WL-derived atom groups in learning. That is a different use of equivalence information from preserving joint placement support during combinatorial growth. Its terminology should not be treated as a proof that finite WL refinement identifies every exact automorphism orbit.²⁵

Bond weights and alternative mappings

AMLGAM estimates bond stabilities with machine learning, minimizes a weighted edit objective through mixed-integer optimization, and then annotates equivalent atoms in a post-processing step. The distinction from the baseline is the role of the weight: AMLGAM's weights define optimization costs, whereas the baseline's WBO values help define compatibility and which exposed source edge is attempted next. Their common use of chemically informed bond weights is established background, not the novel mechanism.²⁶

Latendresse's author presentation of MWED likewise describes bond propensities, assignment constraints, and equivalent-mapping treatment after optimization. This is direct author evidence for those operations, though it does not substitute for an audit of the complete published implementation.²⁷

The original abstracts for Heinonen's A* approach and CLCA identify, respectively, minimum-edit mapping with heuristic search and canonical-label-based approximate common-substructure queries. ReactionMap's original abstract identifies another relevant mapper. Those abstracts are insufficient to exclude every possible internal overlap with the baseline. CLCA's associated Alchemist repository contains prime-label processing code, but inspecting that utility is not equivalent to recovering the complete published CLCA mapping algorithm.^{28,29,30}

Contribution statement and manuscript implications

The evidence supports the following contribution statement:

We introduce an atom-mapping algorithm that grows a common source fragment across compressed candidate placements. Each selected frontier extension is committed when any retained placement supports it; unsupported candidates are removed, while unsupported edges are deferred. Growth continues to saturation before whole-mapping alternatives are scheduled. Whole-island absorption and continuation joins preserve the context needed for subsequent growth, and completed alternatives are represented as correlated mapping families.

This is a description of the specific algorithm, without a universal first-ever assertion. A related-work paragraph can then name the contrasting operations: concrete-pair growth in OpenChemLib and SynKit's approximate matcher; fragment-shape enumeration in FMCS; residual-match commitment in Indigo and RDT; assignment/refinement search in SLAP and SynKit's exact mapper; and constrained partial-map extension in Laffitte and colleagues.

Incremental reuse of embeddings, by itself, is not enough to establish a new matching primitive. If $E(F, L)$ contains every embedding of fragment F under locks L , and extension is complete, then the union of supported one-atom extensions over $E(F, L)$ equals $E(F + x, L)$. Every enlarged embedding restricts to an old embedding, and every valid extension defines an enlarged embedding. The novelty argument must therefore explain the policy that chooses and commits x , and the representation that makes that policy practical.

The existing Golden and 140-case results establish only their documented benchmark outcomes. They do not attribute performance separately to continued growth after uniqueness, collective placement filtering, compression, or state joins. Matched component ablations would substantiate those causal claims. They are future experiments, not results of this literature assessment. The SLAP single-cut sweep remains a supporting constraint-relaxation ablation; it is not the central algorithmic contribution.

The most valuable remaining documentary work is obtaining the original historical growth papers and full methods for CLCA, ReactionMap, and the older graph-naming/cut algorithms. That gap limits claims of universal priority. It does not erase the demonstrated control-flow differences from the directly inspected methods. The present evidence supports proceeding with an algorithm-centered paper whose novelty claim names the coupled transition and state representation precisely.

Sources

The numbered notes give the original locations and the portions used. Repository revisions are fixed where source code supplies the comparison. Review descriptions are identified as such and are not counted as full inspections of the papers they cite.

1. Zou, Y. Continuous-fragment-growth implementation, revision 98b01b175eed31f70d13e7cbf178b80bf07c9e0. [Python growth loop](#); [native loop](#). Direct source inspection.
2. Chen, W. L.; Chen, D. Z.; Taylor, K. T. *Automatic reaction mapping and reaction center detection*. WIREs Computational Molecular Science 3, 560–593 (2013). DOI: 10.1002/wcms.1140. [Full review](#). Historical EC/MCS, isomorphism, A* and optimization sections and references.
3. Akutsu, T.; Nagamochi, H. *Comparison and Enumeration of Chemical Graphs*. Computational and Structural Biotechnology Journal 5, e201302004 (2013). [Original article](#). Atom-mapping review section.
4. Duesbury, E.; Holliday, J.; Willett, P. *Maximum Common Subgraph Isomorphism Algorithms*. MATCH Communications in Mathematical and in Computer Chemistry 77, 213–232 (2017). [Accepted manuscript](#). Sections on subgraph enumeration, McGregor, clique approaches and references.
5. Preciat-Gonzalez, G., et al. *Comparative evaluation of atom mapping algorithms for balanced metabolic reactions: application to Recon 3D*. Journal of Cheminformatics 9, 39 (2017). [Original article](#). Six-method overview and original-method references.
6. Lin, A., et al. *Atom-to-atom Mapping: A Benchmarking Study of Popular Mapping Algorithms and Consensus Strategies*. Molecular Informatics 41, 2100138 (2022). DOI: 10.1002/minf.202100138. [Author manuscript](#). Introduction, mapper descriptions and bibliography.
7. Zou, Y. Frozen implementation, revision 98b01b1. [Support search](#); [extension](#); [whole-mapping branches](#); [family operations](#). See the companion mechanism audit for the detailed key comparison.
8. Actelion/OpenChemLib contributors. [SimilarityGraphBasedReactionMapper.java](#), version 2026.2.0, revision 0754c781c23f7570b13866e202836b5449282ad6. Growth and outer root-sequence selection; not a complete audit of the fallback mapper.
9. SynKit contributors. [approx_mcs.py](#), revision b57f2a39f1807a2682913183b22e1d8d61531bba. [_grow_from_seed](#) and [_approximate_mappings](#), lines 544–686.
10. RDKit contributors. [rdkit.Chem.fmcs.fmcs documentation](#). Documented Python FMCS algorithm, accessed 10 September 2026; not a version-wide audit of the native rdFMCS engine.
11. EPAM/Indigo contributors. [reaction_automapper.cpp](#), revision 8642d73cca2b827a5351d8a8eb21816492381497. Outer permutation loop and [_handleWithProduct](#), lines 260–435.
12. ReactionDecoder contributors. [GameTheoryEngine.java](#) and [GraphMatcher.java](#), revision 0d6632251108e7a8f625d15c60607a136093c4d8. MAX iteration and match commitment/removal; not the complete 2016 source release.

- 13.** Armitage, J. E.; Lynch, M. F. *Automatic detection of structural similarities among chemical compounds*. Journal of the Chemical Society C, 521–528 (1967). [Publisher record](#). DOI: 10.1039/J39670000521. Original metadata/abstract; full algorithm unavailable. Historical mechanism lead from source 4.
- 14.** McCreesh, C.; Prosser, P.; Trimble, J. *A Partitioning Algorithm for Maximum Common Subgraph Problems*. IJCAI (2017), 712–719. [Original paper](#), Algorithm 1.
- 15.** Jaworski, W., et al. *Automatic mapping of atoms across both simple and complex chemical reactions*. Nature Communications 10, 1434 (2019). [Original article](#). Main algorithm description; supplementary internals unresolved.
- 16.** Yan, X.; Han, J. *gSpan: Graph-Based Substructure Pattern Mining*. ICDM (2002). [Author-hosted paper](#), Subprocedure 1.
- 17.** Petersen, J. B. S., et al. *On Finding All Connected Maximum-Sized Common Subgraphs in Multiple Labeled Graphs*. arXiv:2503.22368v2 (2025). [Original manuscript](#), Sections 2–3. The revised version supplies the comparison.
- 18.** SLAP contributors. [core.py](#), revision ea248fd9494f52f4865193e87a98cc92c62b5f9e. LAP, label-state refinement and visited-state operations.
- 19.** SynKit contributors. [exact/branching.py](#) and [exact/enumerate.py](#), revision b57f2a39f1807a2682913183b22e1d8d61531bba. Assignment recursion and symmetry keys.
- 20.** Dechter, R.; Mateescu, R. *AND/OR search spaces for graphical models*. Artificial Intelligence 171, 73–106 (2007). [Author-hosted paper](#). Context-based merging as general search precedent.
- 21.** McKay, B. D.; Piperno, A. *Practical Graph Isomorphism, II*. Journal of Symbolic Computation 60, 94–112 (2014). [Author manuscript](#). Canonicalization and automorphism foundations.
- 22.** González Laffitte, M. E.; Phan, T.-L.; Stadler, P. F. *Extension of partial atom-to-atom maps: uniqueness and algorithms*. Algorithms for Molecular Biology 21, 10 (2026). [Original article](#). Partial-map extension, anchored matching and automorphism relationships.
- 23.** Mann, M., et al. *Atom mapping with constraint programming*. Algorithms for Molecular Biology (2014). [Original article](#). Cyclic reaction-center constraints and graph-matching extension.
- 24.** Ali, M., et al. *Enumeration Approach to Atom-to-Atom Mapping Accelerated by Ising Computing*. Journal of Chemical Information and Modeling (2025). [Full original article](#). DOI: 10.1021/acs.jcim.4c01871. Common-edge-subgraph enumeration and symmetry treatment; no independent implementation certification.
- 25.** Astero, M.; Rousu, J. *Learning symmetry-aware atom mapping in chemical reactions through deep graph matching*. Journal of Cheminformatics 16, 46 (2024). [Original article](#), symmetry identification and atom matching network sections. The method is named AMNet in this article.
- 26.** Litsa, E. E., et al. *Machine Learning Guided Atom Mapping of Metabolic Reactions*. Journal of Chemical Information and Modeling 59, 1121–1135 (2019). DOI: 10.1021/acs.jcim.8b00434. [Author manuscript](#), MILP formulation and alternative-mapping sections, manuscript pp. 8–15.
- 27.** Latendresse, M. *Atom Mapping in MetaCyc and Pathway Tools*. Author presentation (2013). [Slides](#), slides 11–19. Companion paper: Latendresse et al. (2012), DOI: 10.1021/ci3002217; the slides supply the inspected mechanism evidence.
- 28.** Heinonen, M., et al. *Computing Atom Mappings for Biochemical Reactions without Subgraph Isomorphism*. Journal of Computational Biology 18, 43–58 (2011). [Author-institution abstract](#). DOI: 10.1089/cmb.2009.0216. Full original algorithm unresolved.
- 29.** Kumar, A.; Maranas, C. D. *CLCA: Maximum Common Molecular Substructure Queries within the MetRxn Database*. Journal of Chemical Information and Modeling 54, 3417–3438 (2014). [Author-institution abstract](#). DOI: 10.1021/ci5003922. Associated [Alchemist prime-label utility](#); complete CLCA procedure unresolved.
- 30.** Fooshee, D.; Andronico, A.; Baldi, P. *ReactionMap: An Efficient Atom-Mapping Algorithm for Chemical Reactions*. Journal of Chemical Information and Modeling 53, 2812–2819 (2013). [Original abstract](#). DOI: 10.1021/ci400326p. Abstract-level coverage; full-method equivalence not assessed.